

increase the maximum recursion depth to 100 to extend the horn into a tight spiral.

Snowflake

The final fractal, *Snowflake*, is composed of squares laid out in a pentagon pattern. This is similar to the Four Corners fractal, but it uses five evenly spaced recursive squares instead of four:

Python

```
elif DRAW_FRACTAL == 9:
    # Snowflake:
    drawFractal(drawFilledSquare, 200,
        [{'xChange': math.cos(0 * math.pi / 180),
          'yChange': math.sin(0 * math.pi / 180), 'sizeChange': 0.4},
         {'xChange': math.cos(72 * math.pi / 180),
          'yChange': math.sin(72 * math.pi / 180), 'sizeChange': 0.4},
         {'xChange': math.cos(144 * math.pi / 180),
          'yChange': math.sin(144 * math.pi / 180), 'sizeChange': 0.4},
         {'xChange': math.cos(216 * math.pi / 180),
          'yChange': math.sin(216 * math.pi / 180), 'sizeChange': 0.4},
         {'xChange': math.cos(288 * math.pi / 180),
          'yChange': math.sin(288 * math.pi / 180), 'sizeChange': 0.4}])
```

This fractal uses the cosine and sine functions from trigonometry, implemented in Python's `math.cos()` and `math.sin()` functions, to determine how to shift the squares along the x-axis and y-axis. A full circle has 360 degrees, so to evenly space out the five recursive squares in this circle, we place them at intervals of 0, 72, 144, 216, and 288 degrees. The `math.cos()` and `math.sin()` functions expect the angle argument to be in radians instead of degrees, so we must multiply these numbers by `math.pi / 180`.

The end result is that each square is surrounded by five other squares, which are surrounded by five other squares, and so on, to form a crystal-like fractal that resembles a snowflake.

Producing a Single Square or Triangle

For completion, you can also set `DRAW_FRACTAL` to 10 or 11 to view what a single call to `drawFilledSquare()` and `drawTriangleOutline()` produce in the turtle